

Tema3: Cifrado simétrico

Docente : Héctor Xavier Limón Riaño
email: hlimon@uv.mx

Introducción

- ▶ Como ya se mencionó antes el cifrado es el elemento principal de seguridad para salvaguardar la confidencialidad de la información
- ▶ El cifrado simétrico es el fundamento de todas las comunicaciones seguras modernas
- ▶ Simétrico significa que la misma llave que se utiliza para cifrar se utiliza también para descifrar (hay un secreto compartido)

AES

- ▶ Advanced Encryption Standard
- ▶ Este nombre es más bien un título
- ▶ Actualmente el algoritmo que tiene este título es Rijndael
- ▶ Para no crear confusión, pensemos que AES es el algoritmo de cifrado

- ▶ Es el algoritmo de cifrado simétrico más utilizado en la actualidad
- ▶ Es el que deberíamos usar por lo general
- ▶ Lo utiliza TLS (HTTPS) y las funciones de cifrado de sistemas de archivos de un OS
- ▶ Pero debe saberse utilizar, tiene diferentes modos de operación, algunos de ellos inseguros

AES

- ▶ Existen variantes, como AES-128 que utiliza una llave de 16 bytes
- ▶ Una llave con contenido aleatorio es una llave válida

```
1 import os
2 llave = os.urandom(16)
```

AES modo ECB

- ▶ Este es un modo de operación simple de AES
- ▶ Nunca debe usarse porque es inseguro

```
1  from cryptography.hazmat.primitives.ciphers import Cipher,
2      algorithms, modes
3  from cryptography.hazmat.backends import default_backend
4  import os
5  key = os.urandom(16)
6  aesCipher = Cipher(algorithms.AES(key),
7      modes.ECB(),
8      backend=default_backend())
9  aesEncryptor = aesCipher.encryptor()
10 aesDecryptor = aesCipher.decryptor()
11 aesEncryptor.update(b'datos a cifrar')
12 aesDecryptor.update(datosCifrados)
13 aesEncryptor.finalize()
14 aesDencryptor.finalize()
```


Ejemplo

- ▶ Modificar el algoritmo de César para utilizar AES ECB, lidiar con los problemas de padding

Cifradores de bloque y de flujo

- ▶ AES puede operar como cifrador de flujo, basta con usar un modo de AES que trabaje con un tamaño de bloque de 1
- ▶ En general, es más eficiente cifrar en bloques que en flujo

Otros algoritmos

- ▶ RC4 es un cifrador de flujo que ha caído en desuso debido a que tiene vulnerabilidades
- ▶ Twofish es otro cifrador de bloque, el cual fue uno de los algoritmos candidatos para convertirse en AES
- ▶ DES y 3DES son cifradores de bloque muy populares en el pasado, pero actualmente se encuentran en desuso y deben evitarse

Modos de operación de AES

Introducción

- ▶ AES tiene diversos modos de operación, en este tema nos concentraremos en 3:
 - ▶ Electronic code book (ECB) (NO USAR NUNCA)
 - ▶ Cipher block chaining (CBC)
 - ▶ Counter mode (CTR)
- ▶ No son todos los modos, CBC y CTR se siguen usando pero un modo más nuevo llamado GCM es el que se prefiere, dicho modo se verá en otros temas del curso

ECB

- ▶ Es el modo "crudo" (raw) de AES
- ▶ Se verá por fines didácticos, pero nunca debe usarse en producción
- ▶ En este modo AES trata cada bloque (16 bytes comúnmente) de forma independiente
- ▶ Cifra cada bloque de la misma forma

ECB

- ▶ Electronic code book hace referencia a tiempos anteriores donde se utilizaban libros de códigos
- ▶ ECB es análogo a tener un gran diccionario de texto plano a texto cifrado. Cualquier texto plano de 16 bytes tiene 16 bytes correspondientes de salida

ECB

- ▶ La debilidad de ECB yace en el hecho de que cada bloque es tratado independientemente
- ▶ Esto abre la puerta a posibles ataques, por ejemplo: dados dos mensajes con los mismos encabezados (por ejemplo encabezados del mismo protocolo), cifrados con la misma llave, es posible intercambiar bloques entre los mensajes para generar uno nuevo, sin necesidad de conocer la llave

ECB

Considera la siguiente estructura de mensaje (tipo email):

- ▶ De:
- ▶ Para:
- ▶ Asunto:
- ▶ Fecha:
- ▶ Cuerpo

Genera dos mensajes con esta estructura, cófralos con ECB y analiza el contenido binario

ECB

- ▶ Con la vulnerabilidad anterior, un atacante podría crear un ataque de **texto plano escogido**
- ▶ Este ataque consiste en tener muchos mensajes con una estructura parecida (obtenida de alguna forma) y luego engañar a la víctima para que conteste un mensaje falso, cuya respuesta es predecible. Así el atacante puede armar un mensaje que parece legítimo sin descifrar la información ni conocer la llave
- ▶ Nunca debes asumir que el atacante no puede hacerse con cierta información, un método de cifrado debe ser seguro aunque el atacante conozca el método y tenga muestras de mensajes, sólo se debe perder la seguridad si se compromete la llave

Ejercicio

- ▶ Para tener una última prueba de que ECB no es seguro
- ▶ Considera un archivo BMP cualquiera (lo puedes generar con gimp por ejemplo)
- ▶ En este caso la cabecera de un BMP es de 54 bytes
- ▶ Cifra un archivo BMP que contiene un mensaje de texto mediante AES ECB, ignorando los primeros 54 bytes, de tal forma que el archivo cifrado siga siendo un archivo BMP válido
- ▶ ¿Observas algo raro en el archivo cifrado?
- ▶ Puedes utilizar el código realizado por el maestro que considera los problemas de padding, por ejemplo:
 - ▶ Puedes modificar el código del maestro para no cifrar los primeros 54 bytes y ajustar el cálculo de padding, sólo será necesario cifrar, no se necesita descifrar
 - ▶ Puedes quitar los primeros 54 bytes del BMP guardar el resto en un archivo, cifrar el resto con el código tal cual del maestro y luego pegarle los 54 bytes que quitaste

Seguridad en cifrado simétrico

Para tener un cifrador seguro y efectivo se necesita:

1. Cifrar el mismo mensaje de forma diferente cada vez que se haga, aunque se use la misma llave
2. Eliminar patrones predecibles entre los bloques

Seguridad en cifrado

- ▶ Para resolver el primer problema un truco efectivo es que nunca se use dos veces el mismo texto plano, para así tener también un texto cifrado diferente
- ▶ Lo anterior se logra con un vector de inicialización (IV)
- ▶ Un IV es usualmente una cadena aleatoria que se utiliza como tercera entrada del proceso de cifrado (las otras entradas son el texto plano y la llave)
- ▶ La forma exacta en que se usa el IV depende del modo de operación, pero sirve en general para prevenir que un texto plano produzca siempre el mismo texto cifrado

Seguridad en cifrado

- ▶ A diferencia de la llave, el IV es público.
- ▶ Se puede asumir que el atacante puede conocer el IV
- ▶ La presencia del IV no ayuda a la privacidad del mensaje sino a que los mensajes cifrados no sean repetibles, para que sea más complicado encontrar patrones en dichos mensajes

Seguridad en cifrado

- ▶ Para el segundo problema (no tener patrones entre bloques cifrados) se requieren formas para cifrar el mensaje en su totalidad, no sólo cada bloque de forma independiente (como lo hace ECB)
- ▶ Los detalles de cada solución antes mencionada se especifican en el modo de operación de AES

Seguridad en cifrado

- ▶ Los algoritmos de hash seguros tienen la propiedad de avalancha (como se vio en el tema anterior)
- ▶ Los cifradores por bloque deberían tener una propiedad similar: un cambio en un bloque debería afectar el cifrado de los demás bloques
- ▶ Una forma sencilla pero efectiva para lograr lo anterior es utilizando la función XOR, de hecho esta es la forma en que trabaja el modo CBC de AES

CBC: Cipher Block Chaining

- ▶ La idea es: después de cifrar un bloque aplicar la función XOR de dicho bloque con el siguiente bloque no cifrado. El resultado se cifra y es lo que realmente se guarda para ese bloque (luego se verá que se hace con el primer bloque)
- ▶ Para descifrar hay que invertir el proceso: el texto cifrado se descifra y luego la función XOR es aplicada al bloque siguiente
- ▶ De esta forma cada bloque depende del anterior, osea se procesa en cadena
- ▶ Para entender cómo funciona esto se revisará la función XOR

XOR $\oplus$

- ▶ Simbólicamente $\oplus$
- ▶ Es un operador booleano binario (necesita dos operandos)

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

XOR $\oplus$

- ▶ La XOR tiene una propiedad de inversión: la operación XOR es su propio inverso
- ▶ Si se empieza con un número binario A y se le aplica XOR con B , puede recuperarse A aplicando XOR de nuevo con el resultado anterior y B , esto es:
- ▶ $(A \oplus B) \oplus B = A$
- ▶ En python la operación XOR es soportada directamente para números enteros mediante el operador $\wedge$

$$\begin{array}{r} 11011011 \\ \oplus 10110001 \\ \hline 01101010 \\ \oplus 10110001 \\ \hline 11011011 \end{array}$$

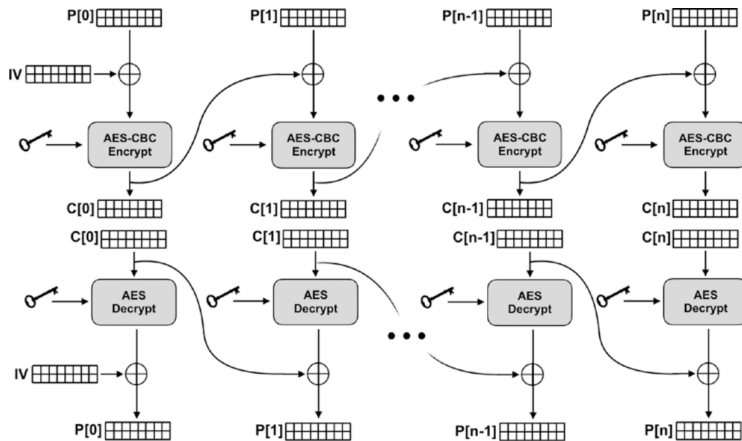
CBC

- ▶ Se aplica XOR a la salida de un bloque con el siguiente bloque de texto plano, esto es:
- ▶ $P'[n] = P[n] \oplus C[n - 1]$, donde:
- ▶ $P[n]$ un bloque n de texto plano
- ▶ $P'[n]$ un bloque n de texto pre-cifrado (sólo es el resultado de XOR, falta aplicarle el algoritmo de cifrado en si)
- ▶ $C[n - 1]$ es el bloque anterior $n - 1$ de texto cifrado (ahora si completamente cifrado).
- ▶ Si a $P'[n]$ lo ciframos, obtenemos $C[n]$

CBC

- ▶ Para regresar al texto plano $P[n]$ basta con realizar el proceso de forma inversa, pues se sabe (por la propiedad de inversión de XOR) que:
- ▶ $P[n] = P'[n] \oplus C[n - 1]$
- ▶ En el caso del primer bloque, al no tener anterior, simplemente se le aplica XOR con el IV (el IV debe ser del tamaño de bloque)
- ▶ Con esto, dados dos mensajes iguales pero con diferente IV, el resultado de cifrar es diferente

CBC



CBC

- ▶ Puede verse que una modificación a un bloque afecta a todos los siguientes, creando un efecto tipo avalancha, no exactamente igual que en hashing, porque los bloques anteriores no se ven afectados
- ▶ Para usar CBC es casi igual que con ECB, sólo hay que agregar el IV

CBC

```
1 from cryptography.hazmat.primitives.ciphers import Cipher,
2     algorithms, modes
3 from cryptography.hazmat.backends import default_backend
4 import os
5 key = os.urandom(32)
6 iv = os.urandom(16)
7 aesCipher = Cipher(algorithms.AES(key),
8                     modes.CBC(iv),
9                     backend=default_backend())
10 aesEncryptor = aesCipher.encryptor()
11 aesDecryptor = aesCipher.decryptor()
```


Padding apropiado

```
1  from cryptography.hazmat.primitives import padding
2  # otros imports
3
4  padder = padding.PKCS7(128).padder()
5  unpadder = padding.PKCS7(128).unpadder()
6
7  m = padder.update(AlgünBinario) #solo llena buffer
8  m = padder.finalize() #aplica padding al resto
9
10 m2 = unpadder.update(AlgünBinario)
11 m2 = unpadder.finalize() #quita el padding
```

Ejercicio

- ▶ Tomando como base el código realizado para ECB, crea una versión con CBC
- ▶ Recibe la llave y el IV como parámetros, codificados en base64
- ▶ En este caso no se manejará manualmente el padding, sino que deberás usar el código visto antes

Seguridad e IV

- ▶ Es muy importante que **NUNCA** se reuse el mismo IV
- ▶ Recordando que el IV sirve para que dos mensajes iguales den como resultado una salida diferente, siempre y cuando no se reuse el IV, sino la salida es la misma
- ▶ Esto evita que un atacante encuentre patrones explotables en una transmisión, sin importar que el IV sea público
- ▶ De hecho lo mejor sería tampoco reusar llaves, pero en ocasiones esto puede ser complejo (el emisor y el receptor necesitan ponerse de acuerdo de nuevo)

CTR

- ▶ Counter mode (en este caso no se usan las siglas CM)
- ▶ Tiene diversas ventajas sobre CBC y es más fácil de entender
- ▶ Es un cifrador de flujo por lo que no requiere de padding
- ▶ Algo contra intuitivo de este modo es que nunca usa AES para cifrar o descifrar los datos
- ▶ Este modo genera un **key stream** que es de la misma longitud del texto plano y luego aplica XOR para combinarlos

CTR

- ▶ Como ya se dijo XOR ayuda a enmascarar texto plano a partir de la combinación con datos aleatorios
- ▶ De hecho, enmascarar de esta manera es una forma real de cifrado llamada **one-time pad (OTP)**
- ▶ Un problema de OTP es que requiere que la llave sea del mismo tamaño que el texto plano (imagina un texto plano de 1TB)
- ▶ De cualquier forma, enmascarar texto plano mediante XOR y datos aleatorios es una forma válida y segura de cifrar información

CTR

- ▶ AES-CTR es similar a OTP
- ▶ Pero en vez de requerir una llave del mismo tamaño que el texto plano, utiliza AES y un contador para generar un key stream de prácticamente cualquier longitud arbitraria a partir de una llave AES de al menos 128 bits

CTR

- ▶ CTR usa AES para cifrar un contador de 16 bytes, lo cual genera los primeros 16 bytes del key stream
- ▶ Para obtener los siguientes 16 bytes, CTR incrementa el contador en uno y vuelve a cifrar con AES y así sucesivamente
- ▶ A la par que se genera el key stream, se aplica XOR a los bytes correspondiente de texto plano
- ▶ Si la longitud del texto plano no es un múltiplo de 16, no importa: los últimos 16 bytes del key stream se truncan para corresponderse con la longitud de lo que falta del texto plano (por eso no se requiere de padding)

CTR

- ▶ Aunque el contador cambie muy poco cada vez (a veces sólo un bit), AES tiene buenas propiedades de avalancha por bloque
- ▶ Así cada bloque del key stream parece completamente diferente del anterior y el key stream en su totalidad parece datos aleatorios

CTR

- ▶ Como puede entenderse, la aleatoriedad es una propiedad de suma importancia en criptografía, un atacante no debería poder encontrar patrones repetibles
- ▶ CTR sólo requiere que la llave de AES sea aleatoria (nunca uses algo que no sea aleatorio)
- ▶ En el curso se ha usado `os.urandom` como fuente de datos aleatorios, en producción es importante analizar si se requiere de otra fuente

CTR

- ▶ Hasta hora lo que se ha dicho de CTR se puede expresar como: $C[n] = P[n] \oplus n_k$ donde:
 - ▶ $C[n]$ representa el bloque n cifrado
 - ▶ $P[n]$ representa el bloque n de texto plano
 - ▶ n_k representa el índice n de bloque cifrado mediante la llave k

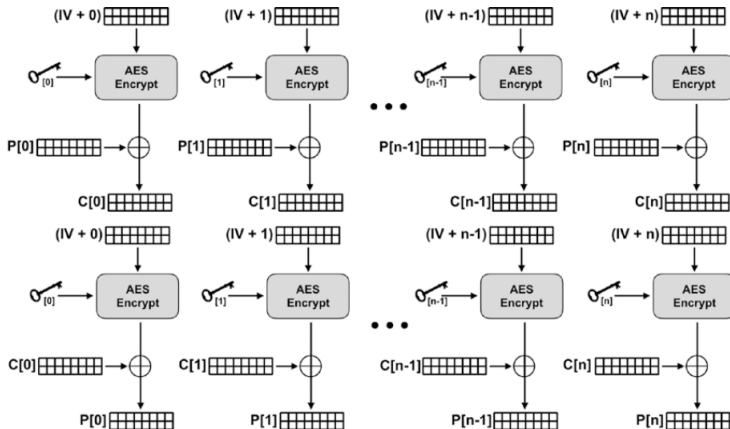
CTR

- ▶ Un problema con la formula anterior es que uno no quisiera empezar siempre con el mismo valor de contador (otra vez por aquello de los patrones repetibles), para esto se utiliza el IV (que hasta ahora estaba olvidado):
- ▶ $C[n] = P[n] \oplus (IV + n)_k$
- ▶ En el contexto de CTR al IV también puede llámesele **nonce**

CTR

- ▶ Para descifrar: $P[n] = C[n] \oplus (IV + n)_k$

CTR



CTR

- ▶ Felizmente los cifradores de flujo no requieren de padding
- ▶ Es bastante simple sólo aplicar XOR a un bloque parcial, descartando las partes del key stream que no se necesitan
- ▶ En general este modo es simple pero mantiene todas las propiedades seguras de CBC: utiliza el IV para que cada salida sea impredecible; y gracias a que el contador se incrementa y a que AES tiene propiedad de avalancha, no hay patrones entre bloques

CTR

- ▶ Un plus es que en CTR cada bloque se puede cifrar de forma independiente (a diferencia de CBC donde se requiere cifrar en orden), esto quiere decir que CTR puede paralelizarse, siendo posible grandes ganancias de eficiencia
- ▶ En general se debe preferir CTR sobre CBC ya que incluso en ciertas circunstancias es más seguro

CTR

```
1  from cryptography.hazmat.primitives.ciphers import Cipher,
2      algorithms, modes
3  from cryptography.hazmat.backends import default_backend
4  import os
5
6  key = os.urandom(32)
7  nonce = os.urandom(16)
8  aes_context = Cipher(algorithms.AES(key),
9                      modes.CTR(nonce),
10                     backend=default_backend())
11
12  encryptor = aes_context.encryptor()
13  decryptor = aes_context.decryptor()
```


- ▶ Como hiciste con CBC, crea una versión del programa que cifra y descifra cualquier archivo, pero esta vez con CTR
- ▶ Esta versión es muy sencilla ya que no se requiere manejar padding

Preocupaciones de seguridad

Manejo de llaves e IV

- ▶ El manejo incorrecto de llaves e IVs es una fuente común de problemas
- ▶ **Nunca** se debe reusar el mismo par de llave e IV
- ▶ Con CBC el reusar pares le brinda a los atacantes la posibilidad de establecer patrones y determinar los posibles mensajes que se están enviando

Manejo de llaves e IV

- ▶ En CTR el reuso de llave-IV tiene consecuencias muy graves
- ▶ Si el atacante conoce el texto plano porque es predecible y se reusa siempre la misma llave e IV, entonces es posible hacer lo siguiente:
 - ▶ $C \oplus P = K$
 - ▶ Esto es: dado el contenido cifrado y el texto plano conocido, es posible recuperar el key stream

Manejo de llaves e IV

- ▶ ¿Y que importa que se pueda saber el key stream si ya se tiene el texto plano?
- ▶ Bajo muchas circunstancias el atacante puede saber una parte o todo el contenido de un mensaje
- ▶ Considerar el siguiente ejemplo: imagina una tienda online donde las transacciones de compra sólo usan AES CTR y siguen un formato XML
- ▶ Un atacante que conoce el formato podría generar una transacción donde supiera todo el contenido del mensaje

Manejo de llaves e IV

<XML>

<CreditCardPurchase>

<Merchant>Acme Inc</Merchant>

<Buyer>John Smith</Buyer>

<Date>01/01/2001</Date>

<Amount>\$100.00</Amount>

<CCNumber>555-555-555-555</CCNumber>

</CreditCardPurchase>

</XML>

Manejo de llaves e IV

- ▶ La tienda genera un mensaje como el anterior, lo cifra y lo manda al banco
- ▶ Para poderse comunicar, el banco y la tienda comparte una llave
- ▶ Si el programador fue descuidado y negligente pudo haber establecido la llave e IV de forma estática en el código (hard-coded)

```
1  # NUNCA hacer esto
2  preshared_key = bytes.fromhex('00112233445566778899AABBCCDDEEFF')
3  preshared_iv = bytes.fromhex('00000000000000000000000000000000')
```

Manejo de llaves e IV

- ▶ Si un atacante intenta crackear el sistema, puede gastar unos \$100.00 dolares en la tienda, esnifar el tráfico e interceptar el mensaje transmitido al banco
- ▶ Si el atacante procede de esta manera, será capaz de conocer todo el mensaje plano (sabe quien hizo la compra, de cuánto fue, la fecha y la tarjeta de crédito)
- ▶ Con esto el atacante recrea el texto plano, le aplica XOR al texto cifrado y recupera el key stream
- ▶ Como el programador fue descuidado, todos los mensajes se cifran con el mismo key stream (o una variante por las posibles diferencias de longitud)
- ▶ ¡El atacante ya puede robar todos los datos de tarjeta de los clientes de la tienda!

Ejemplo

- ▶ Reproducir el escenario de ataque anterior
- ▶ Será necesario utilizar el código de cifrado de archivos mediante CTR, pero hacer hardcoding de la llave e IV
- ▶ Será necesario crear una función que recibe dos cadenas binarias y les aplica XOR

Manejo de llaves e IV

- ▶ Incluso si el atacante no conoce ninguna parte del texto plano y no puede recuperar el key stream, reusar llaves e IVs es inseguro
- ▶ Se puede aplicar el siguiente truco con dos mensajes cifrados con la misma llave e IV (para CTR):

$$c_1 = m_1 \oplus K$$

$$c_2 = m_2 \oplus K$$

$$c_1 \oplus c_2 = (m_1 \oplus K) \oplus (m_2 \oplus K)$$

$$c_1 \oplus c_2 = m_1 \oplus m_2 \oplus K \oplus K$$

$$c_1 \oplus c_2 = m_1 \oplus m_2$$

Manejo de llaves e IV

- ▶ ¿Y de que sirve el XOR de dos mensajes de texto plano? ¿Es legible?
- ▶ Depende, Los mensajes de texto a menudo tienen estructura y cabeceras comunes
- ▶ Los datos privados pueden ser extraída o adivinada
- ▶ Cualquiera dos partes de los mensaje que hagan match van a dar 0 con XOR, pudiendo saber donde dos mensajes son iguales y donde difieren

Manejo de llaves e IV

- ▶ Tomando como ejemplo el XML visto antes, si el atacante tiene la suerte de encontrar dos mensajes de diferentes clientes con una longitud de nombre igual, entonces por ejemplo, los datos del monto de compra se van a alinear
- ▶ Este campo da mucha información cuando se tiene el resultado del XOR de dos números, ya que los caracteres que puede tener son limitados (0-9 y .) y las combinaciones de XOR también lo son
- ▶ Por ejemplo, sólo los pares 8 y 7 (valores ASCII 55 y 56) y 6 y 9 (valores ASCII 54 y 57) pueden producir como resultado 15
- ▶ A partir de este principio el atacante puede adivinar más información mediante prueba y error de combinaciones

Manejo de llaves e IV

- ▶ El mal manejo de llaves e IV es más común de lo que parece
- ▶ Sobre todo el principio de no repetir un par llave-IV debe mantenerse siempre
- ▶ Incluso en mensajes entre dos entidades, debe haber un par para enviar mensajes y otro par para recibir mensajes, siendo una llave diferente de envío y recepción (la cual usualmente se reusa en cada transacción) y un IV diferente en cada mensaje
- ▶ Sobre todo el IV siempre debe ser diferente en cada mensaje
- ▶ Debido a la complejidad de intercambiar llaves, las llaves suelen generarse sólo una vez por sesión

Explotación de maleabilidad

- ▶ Algunos aspectos de la criptografía pueden ser contra-intuitivos, por ejemplo: un atacante puede no ser capaz de leer un mensaje cifrado, pero aun así puede alterarlo en formas que tengan sentido
- ▶ Esto se mostró brevemente antes como una vulnerabilidad de ECB
- ▶ Como ya se dijo antes, el cifrado protege la confidencialidad, pero no la integridad

Explotación de maleabilidad

- ▶ Considerando de nuevo CTR: dado que es un cifrador de flujo con independencia entre cada byte (a diferencia de CBC) , el atacante puede cambiar cualquier byte sin afectar a los demás, a esto se le conoce como maleabilidad La maleabilidad puede ser aprovechada por atacantes para realizar ataques por ejemplo de spoofing
- ▶ A continuación se presenta un ejemplo

Explotación de maleabilidad

- ▶ Considerando de nuevo el ejemplo del XML de la tienda en línea
- ▶ Cada mensaje empieza siempre con:

```
1      <XML>
2          <CreditCardPurchase>
3              <Merchant>Acme Inc</Merchant>
```

- ▶ Un atacante puede conocer al menos esta parte del texto plano pues siempre es el mismo y pudo haberse filtrado de otra forma

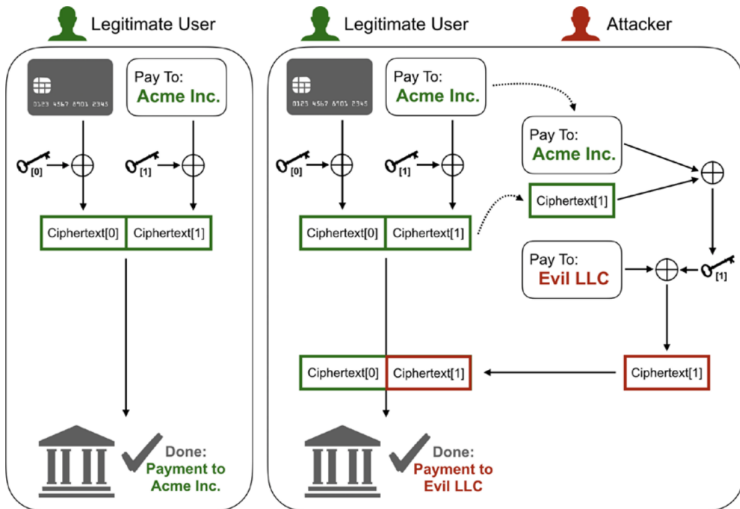
Explotación de maleabilidad

- ▶ Aunque no se reuse la misma llave e IV, si el atacante conoce parte del texto plano, puede recuperar el key stream que se corresponde a ese texto plano
- ▶ Obviamente este key stream no servirá para revelar información confidencial, pues los otros mensajes usan otro par llave-IV
- ▶ Sin embargo, un atacante puede hacer un ataque MITM (Man In The Middle) donde intercepte el mensaje, derive el key stream de la parte conocida del mensaje, lo remplace (como se ve a continuación), aplique XOR a la parte falsa con el keystream derivado y envíe el mensaje a la víctima, en este caso realizando un spoofing

```
<XML>
  <CreditCardPurchase>
    <Merchant>Evil LLC</Merchant>
```

Explotación de maleabilidad

- Esto funciona porque el texto falso es de la misma longitud que el texto original



Tarea

- ▶ Reproduce el ataque anterior con el archivo XML de ejemplo
- ▶ Cifra un archivo XML verdadero mediante CTR e identifica los bytes de ese mensaje cifrado que representan la cabecera conocida
- ▶ Extrae el key stream de esa cabecera
- ▶ Remplaza con el texto falso de la misma longitud del original y cífralo con el keystream
- ▶ Guarda el resultado manipulado y luego descífralo para comprobar que el ataque se realizó bien

Ataques de padding

- ▶ CBC es menos maleable que CTR, sin embargo, también tiene vulnerabilidades
- ▶ De hecho una de las primeras versiones de SSL fue vulnerada debido a defectos de maleabilidad de CBC
- ▶ El problema en este caso estaba en el padding, permitiendo un ataque llamado **ataque de padding oráculo**

Ataques de padding

- ▶ El ataque era posible debido a un algoritmo débil de padding
- ▶ Dicho algoritmo es prácticamente igual al discutido en clase (cuando se vio ECB), con la diferencia de que el último bloque indica los bytes de padding sin tomar en cuenta el último byte y que los bytes de padding son el byte 0, en vez del carácter 0
- ▶ El padding siempre se agrega, incluso cuando no es necesario (osea que el último byte sería 15)
- ▶ En este esquema sólo el último byte importa, todos los demás bytes de padding son desechables

Ataques de padding

- ▶ Si el padding no era necesario (porque el mensaje era múltiplo de 16), en CBC el último bloque es el más maleable, dado que su manipulación no tiene impacto en otros bloques y es predecible
- ▶ Osea que todo el bloque se puede cambiar por cualquier cosa, siempre y cuando el último byte se descifre al tamaño de padding (15 para este padding)
- ▶ Basta con seleccionar un valor de byte final y hacer a lo mucho 256 intentos (los posibles valores de un byte) para obtener un mensaje que sea válido

Ataques de padding

- ▶ SSL V3 tiene una vulnerabilidad que permite saber si un mensaje tiene un padding adecuado o no, haciendo posible para el atacante determinar si sus ataques de maleabilidad están produciendo mensajes con padding correcto
- ▶ En este sentido el servidor SSL es un oráculo que le está diciendo al atacante si está haciendo bien las cosas
- ▶ Mediante este procedimiento, es posible recuperar el último byte del penúltimo bloque sin necesidad de saber la llave (es necesario aplicar operaciones de XOR en el orden correcto)
- ▶ A partir de este byte descifrado, el atacante puede descifrar los demás en cadena realizando intercambios

Seguridad

- ▶ Los ataques anteriores de maleabilidad funcionan debido a que el cifrado sólo provee de confidencialidad
- ▶ El cifrado debe complementarse también con hashing para tener integridad